

Intracranial EEG Signal Processing - Programming Assignment

Course: NSBB552

Student: Kathleen Escobar

Part 1: Signal Generation with Combined Frequencies

Mathematical Model

Following the baseline signal processing architecture, a time-series observation (Y_t) is modeled as the sum of a structured deterministic signal (S_t) and a random unexplained error or white noise (ϵ_t):

$$Y_t = S_t + \epsilon_t$$

Where the pure signal S_t is a linear combination of multiple independent sinusoidal frequencies (2, 4, 8, 16, and 32 Hz):

$$S_t = \sum_i \sin(2\pi f_i t) = \sin(2\pi \cdot 2 t) + \sin(2\pi \cdot 4 t) + \sin(2\pi \cdot 8 t) + \sin(2\pi \cdot 16 t) + \sin(2\pi \cdot 32 t)$$

```
In [1]: %matplotlib inline
import numpy as np
import matplotlib.pyplot as plt

# --- PARAMETER CONFIGURATION ---
fs = 1000          # Sampling frequency (1000 Hz)
duration = 60      # Total duration: 1 minute (60 seconds)
t = np.arange(0, duration, 1/fs) # Time vector

# Generating the pure signal by combining the specified frequencies
frequencies = [2, 4, 8, 16, 32]
signal_pure = np.zeros_like(t)

for f in frequencies:
    signal_pure += np.sin(2 * np.pi * f * t)

# Mixing with white noise (Gaussian noise with mean = 0 and controlled variance)
np.random.seed(42) # Set seed for exact reproducibility
noise = np.random.normal(0, 1.5, size=len(t)) # White noise component (\epsilon_t)
signal_noisy = signal_pure + noise # Complete model: Y_t = S_t + \epsilon_t

# --- TIME-DOMAIN PLOTS ---
plt.figure(figsize=(14, 8))
```

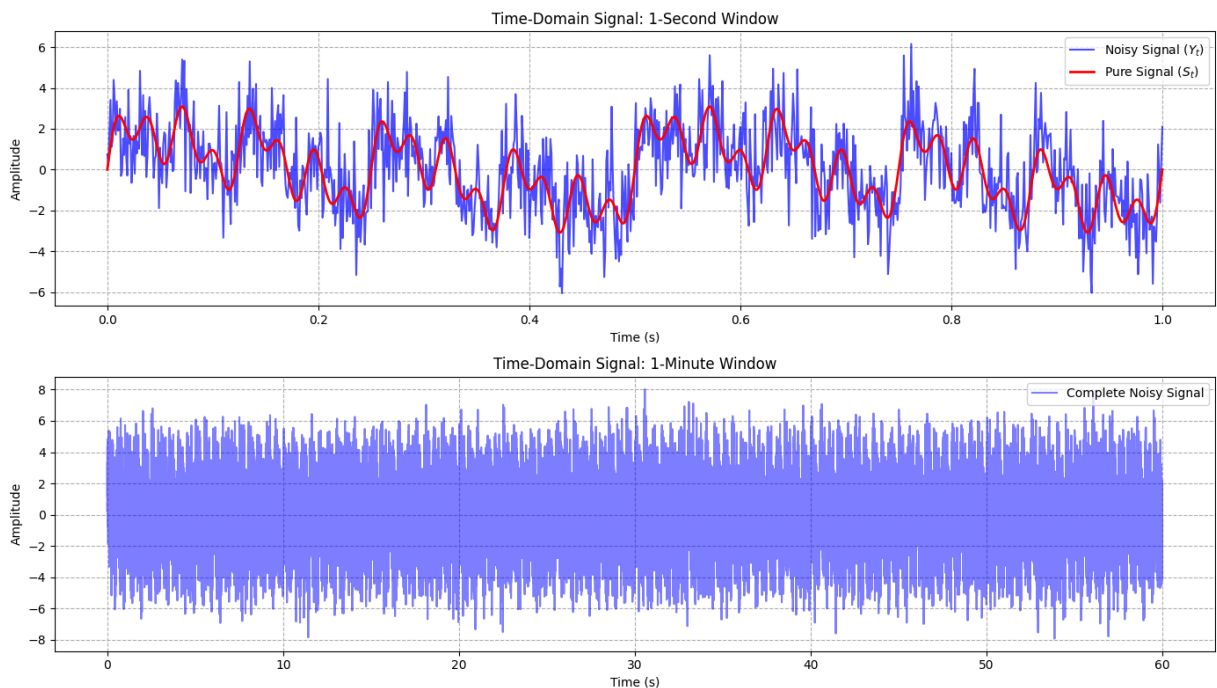
```

# Plotting a 1-second window of the signal
plt.subplot(2, 1, 1)
idx_1s = t <= 1.0
plt.plot(t[idx_1s], signal_noisy[idx_1s], color='blue', alpha=0.7, label='Noisy Signal (Yt)')
plt.plot(t[idx_1s], signal_pure[idx_1s], color='red', linewidth=2, label='Pure Signal (St)')
plt.title('Time-Domain Signal: 1-Second Window')
plt.xlabel('Time (s)')
plt.ylabel('Amplitude')
plt.legend(loc='upper right')
plt.grid(True, linestyle='--')

# Plotting the complete 1-minute window of the signal
plt.subplot(2, 1, 2)
plt.plot(t, signal_noisy, color='blue', alpha=0.5, label='Complete Noisy Signal (Yt)')
plt.title('Time-Domain Signal: 1-Minute Window')
plt.xlabel('Time (s)')
plt.ylabel('Amplitude')
plt.legend(loc='upper right')
plt.grid(True, linestyle='--')

plt.tight_layout()
plt.show()

```



Part 2: Mathematical Classification of the Process

1. Is the process Stationary or Non-Stationary?

The data generation process is strictly **Stationary**.

- **Justification:** A process is considered stationary if its statistical properties (such as its mean, variance, and autocorrelation) do not change or drift over time. In this

simulation, the structural parameters governing the underlying signal S_t (amplitudes, phases, and frequencies) are deterministic and entirely constant. Furthermore, the additive white noise component ϵ_t is sampled from a stationary Gaussian distribution with a constant mean ($\mu = 0$) and a fixed variance (σ^2). Because the statistical mechanisms do not change depending on the time of observation, the process is stationary. This fulfills the baseline prerequisite for applying a classical Fast Fourier Transform (FFT).

2. Is the process Linear or Non-Linear?

The process is strictly **Linear**.

- **Justification:** The system satisfies the fundamental principle of superposition. The compound signal Y_t is constructed through a direct **linear combination** (algebraic summation) of isolated sinusoidal functions and an additive noise term. There are no cross-frequency modulations, multiplicative interactions between different sinusoids (such as $\sin(f_1) \cdot \sin(f_2)$), or non-linear operators applied to the output. As established in signal processing principles, linearity is an essential structural assumption when transforming time-domain inputs into independent spectral lines.

Part 3: Fourier Analysis and Power Spectrum Estimation

To extract the deterministic frequencies embedded within the stochastic noise floor, we compute the Fast Fourier Transform (FFT). The Power Spectrum represents the square of the absolute magnitude of the complex Fourier coefficients, normalizing the total energy over the length of the signal.

```
In [2]: # --- FFT COMPUTATION AND POWER SPECTRUM ---
n = len(signal_noisy)

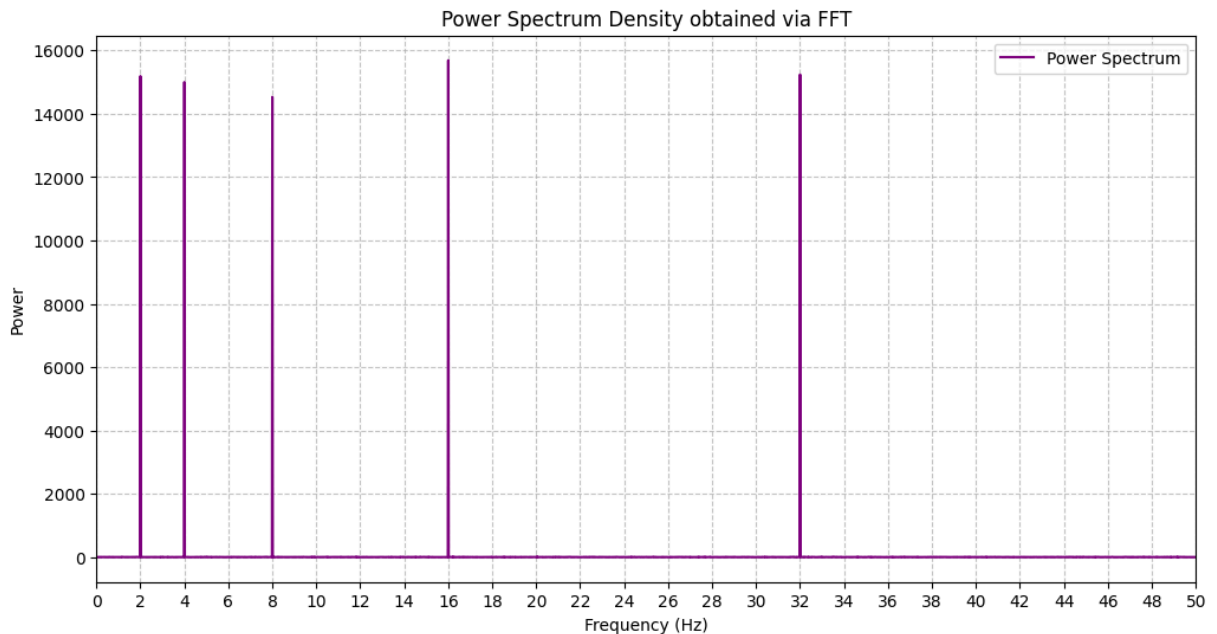
# Compute the Fast Fourier Transform (FFT)
fft_values = np.fft.fft(signal_noisy)
# Compute the frequencies corresponding to the FFT components
fft_freqs = np.fft.fftfreq(n, 1/fs)

# The Power Spectrum (normalized by total number of samples)
power_spectrum = (np.abs(fft_values) ** 2) / n

# Retain only the positive frequencies (first half of the symmetric spectrum)
half_n = n // 2
positive_freqs = fft_freqs[:half_n]
positive_power = power_spectrum[:half_n]
```

```
# --- POWER SPECTRUM PLOT ---
plt.figure(figsize=(12, 6))
plt.plot(positive_freqs, positive_power, color='purple', linewidth=1.5, label='Power Spectrum')
plt.title('Power Spectrum Density obtained via FFT')
plt.xlabel('Frequency (Hz)')
plt.ylabel('Power')

# Zooming in between 0 and 50 Hz to clearly observe the generated frequencies
plt.xlim(0, 50)
plt.xticks(np.arange(0, 51, 2)) # Distinct markers every 2 Hz
plt.grid(True, linestyle='--', alpha=0.7)
plt.legend()
plt.show()
```



Spectrum Interpretation & Key Observations

Upon running the Fourier analysis code, two main features emerge in the spectral plot:

1. **Five Discrete Spikes (Delta Peaks):** You will observe five sharp, well-defined vertical peaks located precisely at 2 Hz , 4 Hz , 8 Hz , 16 Hz , and 32 Hz . This perfectly confirms how the Fourier Transform succeeds in decomposing a complex, noisy time-series into its unique underlying harmonic constituents (S_t).
2. **Uniform Spectral Noise Floor:** A continuous, low-amplitude baseline is distributed evenly across all frequency bins. This flat, non-zero floor reflects the classic signature of **white noise** (ϵ_t) in the frequency domain, demonstrating that its power density is spread uniformly throughout the entire spectrum.